



The Practical Guide to Reverse Engineering XXL Codebases with Agentic AI

••• **Harshad Nawathe**

Solution Consultant

Sahaj Software





Constraints Dictate Architecture

- In-memory algorithms assume unbounded resources
- Hardware limits eventually shatter pure algorithmic theory
- Sufficient scale forces a pivot: local computation → systemic coordination



APS

- **Acknowledge** — Define and respect the absolute processing limits of your container
- **Partition** — Divide monolithic challenges into isolated, parallelizable workloads
- **Synthesize** — Design robust pipelines to merge distributed outputs into a cohesive state



Reverse Engineering A Legacy Codebase



The Reverse Engineering Prompt

You are a Principal Engineer specializing in Java platforms, distributed systems, and modernization.

Analyze the repository, including source code, configuration, build files, database mappings, integration points, and deployment artifacts, to reconstruct the application's architecture.

Identify business domains, bounded contexts, workflows, and the underlying domain model.

Infer entities, aggregates, value objects, relationships, and ownership boundaries.

Document service responsibilities, dependency flows, and architectural layer interactions.

Map REST, messaging, batch, and internal APIs, including downstream dependencies.

Trace end-to-end execution paths for critical business operations and data transformations.

Analyze persistence patterns, schema mappings, transaction boundaries, and integrations.

Highlight design patterns, framework conventions, extension points, and runtime behavior.

Produce architecture diagrams, relationship models, API inventories, and onboarding docs.



Compaction Kills Context

- Agent fills the context window — silently, without warning
- Compaction occurs: instructions are dropped, focus drifts
- LLM starts *forgetting* — and then starts *inventing*
- Larger context window?
Same problem. Bigger bucket.



Acknowledging the Limits

Context is a Finite, Fragile Resource

- Every LLM context window is **a fixed-size processing container**
— not infinite memory
- **Stateless by nature:**
No memory between turns; context must carry everything



The Four Perils of Context Overload

- **Context Contamination**
Irrelevant tokens corrupt reasoning quality
- **Low Signal-to-Noise Ratio**
Useful signal drowns in boilerplate noise
- **Lost in the Middle**
Facts buried deep in context get silently ignored
- **Attention Dilution**
Too many tokens competing, focus scatters



 **What if the agents never read the code?**



Deterministic Tools for Deterministic Work

- **Cognitive tasks** → **LLM agents**
Semantic reasoning, synthesis, boundary analysis
 - **Deterministic tasks** → **purpose-built tools:**
Parsing, indexing, dependency resolution
-

- LLMs read for *meaning* — not for structure
- Structure is already encoded in build files, LSP servers, symbol indexes
- Don't burn context tokens on work a compiler does for free



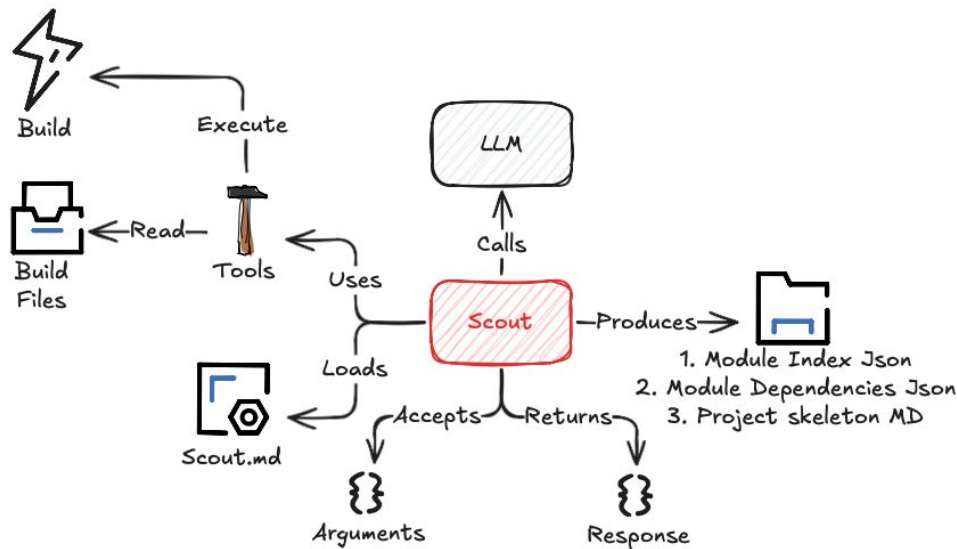
The pipeline



Layer 0

The Scout

- **Mission:**
 - Build the skeleton.
 - Understand the project before anyone reads a line of code.
- **Reads:**
Build files only —
pom.xml, build.gradle, package.json
- **Tools:**
Build tool introspection, glob for module sizing
- **Produces:**
Project skeleton JSON + module summary markdown
- **Reads code?**
Never.

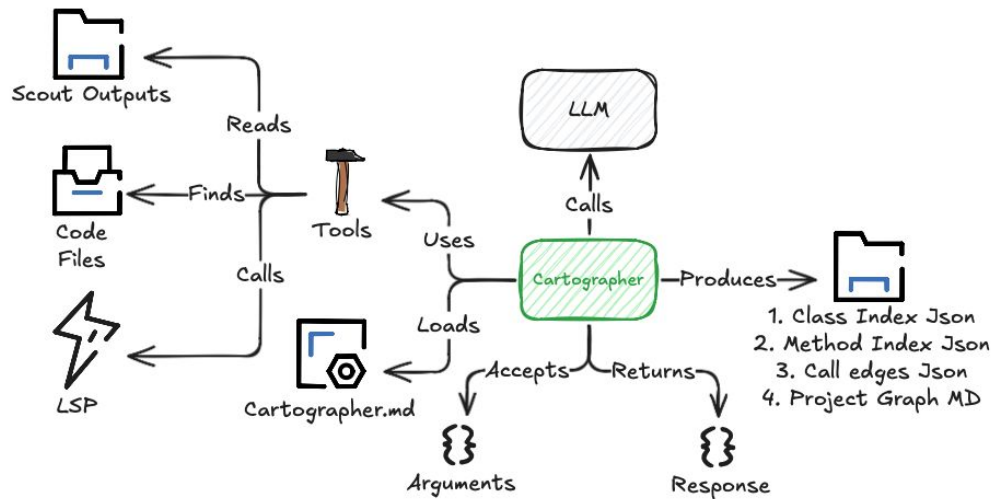




Layer 1

The Cartographers

- **Mission:**
 - Map every symbol in every file. Build the static relationship graph.
- **Reads:**
Scout output — project skeleton JSON
- **Tools:**
LSP servers, symbol indexers, dependency analyzers
- **Produces:**
Symbol index — classes, methods, interfaces, inter-file relationships
- **Reads code?**
Never. Only tool output.

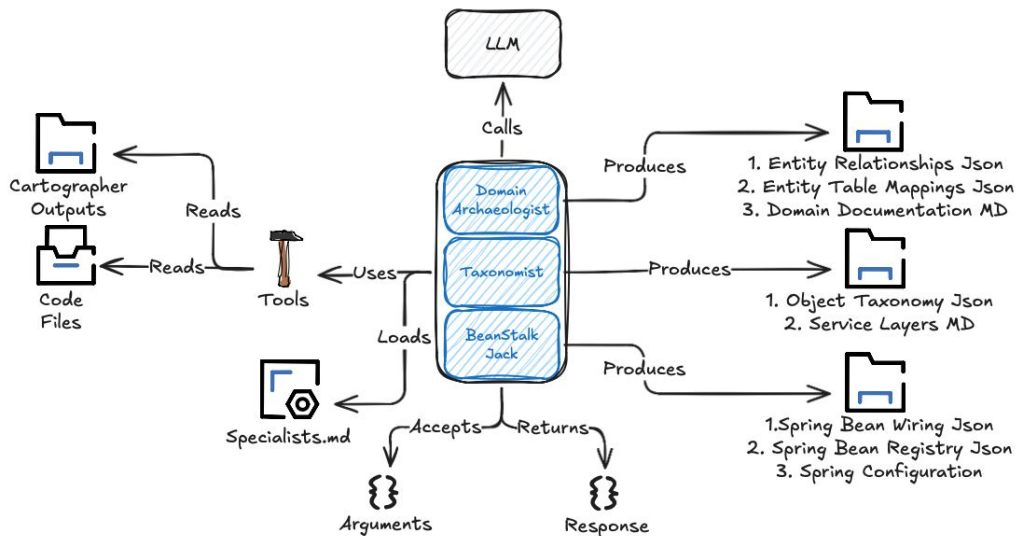




Layer 2

The Specialists

- **Mission:**
Extract domain knowledge from the map.
- **Reads:**
Symbol index from Cartographers
- **Code access:**
Surgical reads only — targeted, bounded, intentional
 - **Archaeologist** — unearths the buried domain model
 - **Taxonomist** — classifies domain concepts and hierarchies
 - **BeanStalkJack** — traces Spring bean wiring and dependency injection
- **Produces:**
Domain artifacts — entity models, bean graphs, concept taxonomies

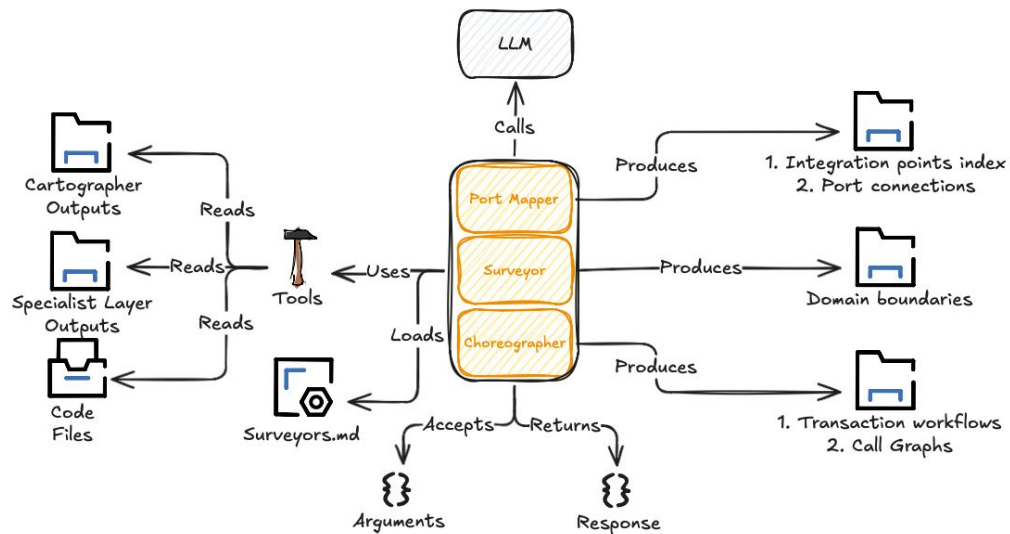




Layer 3

The Surveyors

- **Mission:**
Map the boundaries. Understand how the system moves.
- **Reads:**
All Layer 2 artifacts + symbol index
- **Code access:**
Surgical reads only
 - **Port Mapper** — identifies all inbound and outbound integration points
 - **Surveyor** — draws service and module boundary lines
 - **Choreographer** — traces runtime workflows and call sequences



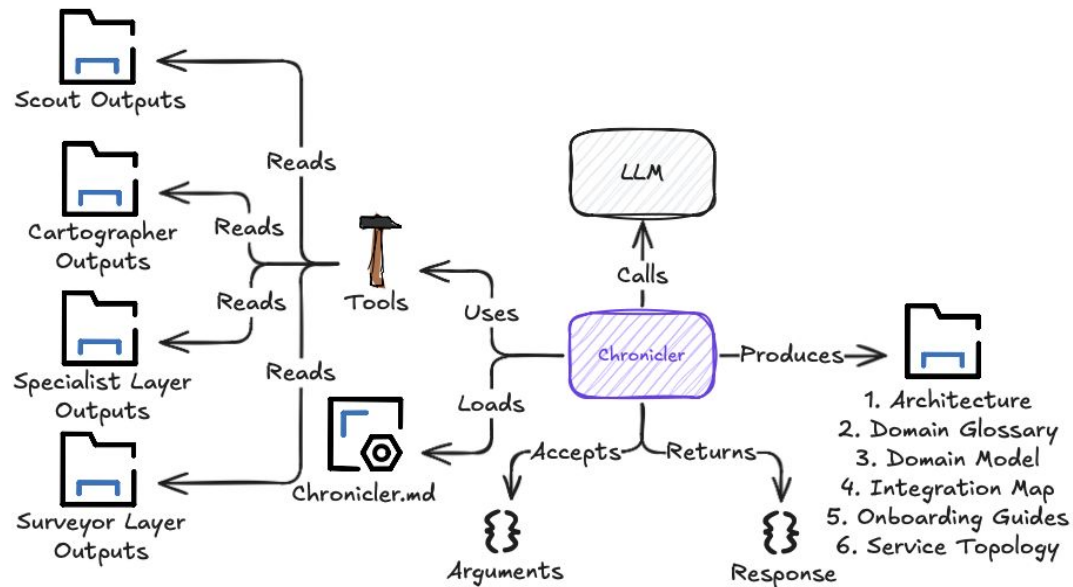
- **Produces:**
Service boundary maps, integration topology, workflow graphs



Layer 4

The Chronicler

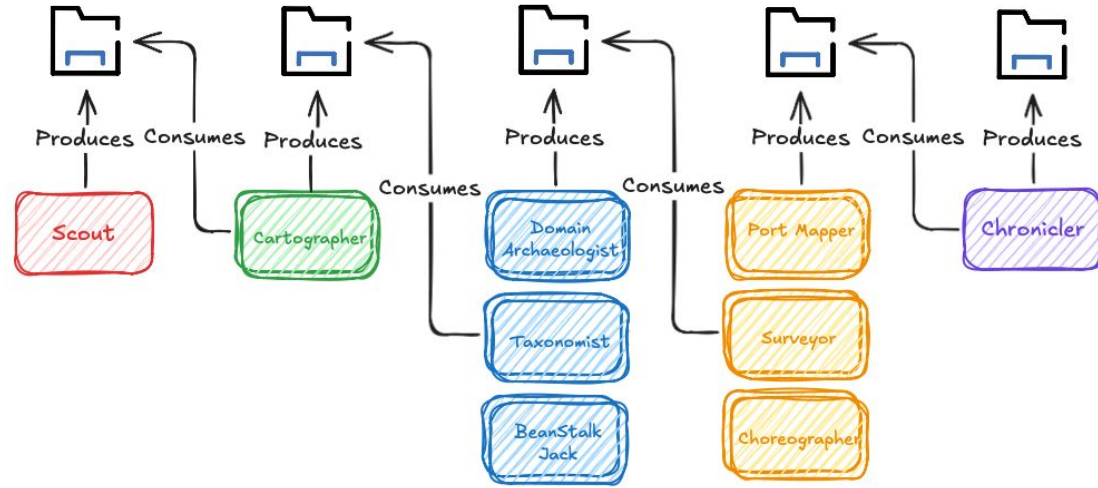
- **Mission:**
Synthesize all layer outputs into the definitive domain documentation
- **Reads:**
All artifacts from Layers 0–3
- **Code access:**
None.
The Chronicler never reads the code.
- **Produces:**
Complete technical domain documentation
— architecture, domain model, service topology, integration map, workflow catalog





The Pipeline

- **Layer 0 — Scout**
project skeleton + module map
- **Layer 1 — Cartographers**
symbol index + static relationships
- **Layer 2 — Specialists**
domain model, bean graph, concept taxonomy
- **Layer 3 — Surveyors**
service boundaries, integration topology, workflows
- **Layer 4 — Chronicler**
complete domain documentation





Who Watches the Workers?

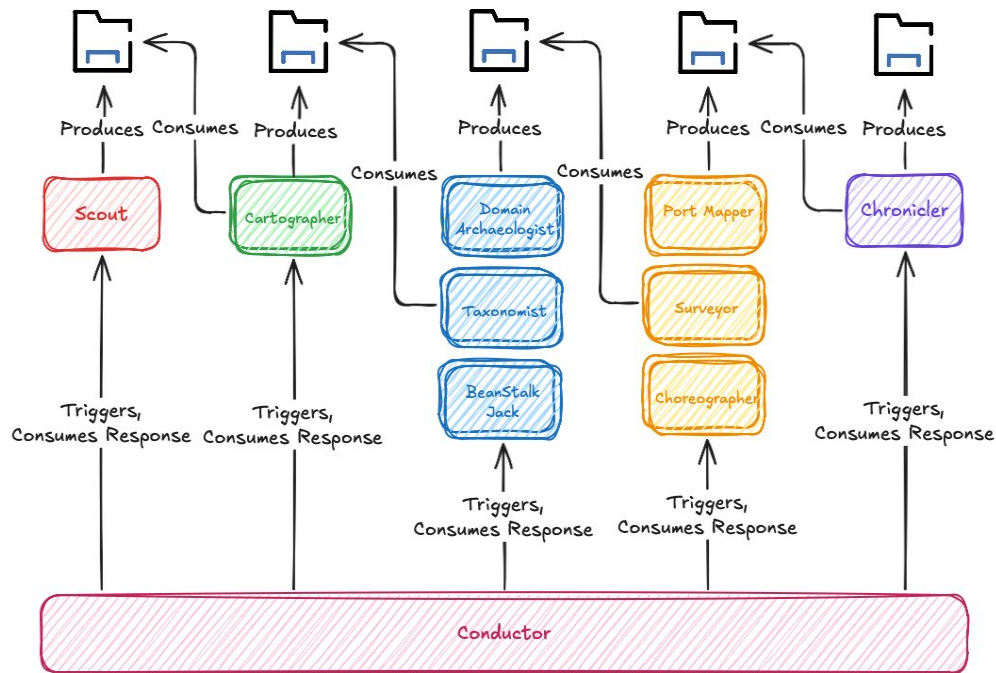
- Who decides how many Cartographers must be spun up?
- What happens when a stage fails with an error?
- How does the pipeline know what runs in parallel and what must wait?



Orchestration Layer

The Conductor

- **Role:**
Coordinates worker agents across all layers.
Manages the pipeline as a Hub — agents are Spokes.
State flows in, instructions flow out.
- **Reads:**
Pipeline state only — what has completed, what failed, what is pending
- **Scale:**
Decides how many workers to spin up.
- **Resilience:**
Handles stage failures — retry, replan, or escalate with context preserved



- **Scheduling:**
Maintains the dependency graph
- **Context:**
The leanest in the entire system — state only, no content



Principles of Context Engineering

- **Progressive Disclosure** —
Reveal information to agents strictly as their role requires. Structure before content. Boundaries before reads.
- **Deterministic Grounding** —
Use fast, deterministic tooling for structural work. Reserve LLM context exclusively for semantic reasoning.
- **Role Isolation** —
One agent, one competency, one narrow context. Specialisation eliminates hallucination and instruction drifts.
- **Subagent Isolation** —
Delegate cognitive tasks to disposable subagent contexts. Keep the orchestration layer uncontaminated.



What the Pipeline Produces

- **Reusable Index** — 10,000+ classes and methods, queryable on demand
- **Documentation Artifacts** — 200+ documents (~15MB), generated from current code
- **Data Layer** — 400+ database tables mapped to entity classes
- **API Catalog** — 80+ REST and 60+ SOAP endpoints, fully indexed
- **Service Boundaries** — 15+ boundaries identified, documented, and ready for decomposition
- **Workflow Library** — 80+ workflow traces with sequence diagrams
- **Architecture** — C4 diagrams at three levels of detail
- **Incremental** — input fingerprints tracked; only changed artifacts are regenerated



Lessons from Production

- **Validate early** —
Trust in generated documentation must be earned, not assumed.
- **Evolve the tools** —
The pipeline is not dogma. If a tool underperforms, replace it. The architecture holds.
- **Evolve freely** —
Decoupled layers evolve independently. The contract between layers is the only thing that must be honoured.
- **Respect the economics** —
Match the instrument to the problem. The full pipeline justifies its cost only when a holistic view is needed.



Putting It All Together

- **Acknowledge** —
We mapped the context window as a finite container and designed every agent to respect that boundary.
- **Partition** —
We decomposed one monolithic problem into five specialised layers, each agent seeing only what it needed.
- **Synthesize** —
The Conductor assembled the work of all agents into a single coherent, living documentation system.

Thank You